Especificação Técnica: MediSystem

Gestão Hospitalar com POO Avançada e Design Patterns

Documentação do Projeto Capstone

21 de janeiro de 2026

Sumário

1 Visão Geral **2**

2 Arquitetura Modular (Definição de Ferramentas) **2**

2.1 1. Núcleo e Acesso (Model & Auth) 2

2.2 2. Módulo de Triagem (Manchester Protocol) 2

2.3 3. Prontuário Eletrônico (Imutabilidade) 3

2.4 4. Módulo Financeiro (Polimorfismo) 4

3 Infraestrutura e Padrões de Projeto **4**

3.1 Tratamento de Erros (Exceptions) 4

3.2 Persistência (Generics) 4

3.3 Singleton (Log) 4

4 Cenários de Uso (Exemplos de Fluxo) **6**

4.1 Cenário 1: Triagem com Prioridade 6

4.2 Cenário 2: Tentativa de Fraude no Prontuário 6

4.3 Cenário 3: Faturamento Recusado 6

1 Visão Geral

O **MediSystem** é um sistema backend robusto projetado para garantir a integridade de dados hospitalares. Diferente do simulador de Kernel (focado em memória e ponteiros), este projeto foca em **Arquitetura, Segurança de Tipos e Manutenibilidade**.

Objetivo Principal: Demonstrar domínio sobre os 4 pilares da POO (Abstração, Encapsulamento, Herança e Polimorfismo) aplicados a regras de negócio complexas.

2 Arquitetura Modular (Definição de Ferramentas)

O sistema deve ser desenvolvido em camadas. Abaixo estão as ferramentas obrigatórias para cada módulo.

2.1 1. Núcleo e Acesso (Model & Auth)

Definição de quem é quem no sistema.

- **Ferramenta:** Herança e Classes Abstratas.
- **Regra:** Crie uma classe base `Pessoa`.
- **Hierarquia:**
  - `Usuario` (Abstrata) → Implementada por `Medico`, `Enfermeiro`, `Recepcionista`.
  - `Paciente` → Herda de Pessoa, mas possui atributos médicos (tipo sanguíneo, alergias).
- **Segurança:** Cada tipo de usuário deve ter um nível de acesso (Enum: `ADMIN`, `MEDICO`, `TRIAGEM`).

2.2 2. Módulo de Triagem (Manchester Protocol)

Gerencia a fila de espera baseada em gravidade, não apenas ordem de chegada.

- **Ferramenta Principal:** `PriorityQueue<Paciente>` (Interface `Queue`).
- **Requisito Técnico:** A classe `Paciente` deve implementar a interface `Comparable<Paciente>`.
- **Lógica de Comparação (compareTo):**
  1. Primeiro critério: **Gravidade** (Vermelho > Amarelo > Verde).
  2. Segundo critério: **Horário de Chegada** (Quem chegou antes).
- **Design Pattern:** Use o padrão **Strategy** para calcular a gravidade baseada nos sintomas (ex: `CalculadoraPrioridade.calcular(Sintomas)`).

2.3 3. Prontuário Eletrônico (Imutabilidade)

O histórico do paciente deve ser inviolável.

- **Ferramenta:** Encapsulamento Rígido e `Collections.unmodifiableList`.
- **Regra de Ouro:** A lista de registros (`List<String> historico`) deve ser `private`.
- **Métodos Permitidos:**
  - `adicionarRegistro(String msg)`: Adiciona uma nova entrada com data/hora.
  - `getHistorico()`: Retorna **apenas uma cópia de leitura** da lista. Nunca retorne a lista original.
- **Proibido:** Criar métodos `setHistorico`, `removerRegistro` ou `editarRegistro`.

2.4 4. Módulo Financeiro (Polimorfismo)

Cálculo de custos dinâmico dependendo do tipo de atendimento.

- **Ferramenta:** Interfaces (`interface IFaturavel`).
- **Contrato:** Todo atendimento deve ter o método `double calcularValorTotal()`.
- **Implementações:**
  - **AtendimentoSUS:** Retorna valor base (tabela fixa).
  - **AtendimentoParticular:** Retorna (valor base * 1.20) + taxas.
  - **AtendimentoConvenio:** Verifica lista de cobertura. Se o procedimento não estiver coberto, lança `CoberturaRecusadaException`.

3 Infraestrutura e Padrões de Projeto

3.1 Tratamento de Erros (Exceptions)

Não use exceções genéricas (`Exception`). Crie sua própria hierarquia.

```java
// Raiz das excecoes do negocio
public class HospitalException extends Exception { ... }

// Filhas especificas
public class PacienteNaoEncontradoException extends
   HospitalException { ... }
public class MedicamentoEsgotadoException extends HospitalException
    { ... }
```

3.2 Persistência (Generics)

Para evitar escrever um código de "Salvar em Arquivo"para cada classe (Medico, Paciente, Remedio), você deve usar **Generics**.

- **Desafio:** Criar uma classe `Repositorio<T>` que saiba salvar qualquer objeto que implemente `Serializable`.
- **Métodos:** `salvar(T objeto)`, `listar()`, `buscarPorId(int id)`.

3.3 Singleton (Log)

Garanta que haja apenas uma instância do gerenciador de Logs no sistema inteiro.

```java
public class SistemaLog {
    private static SistemaLog instancia;
    private SistemaLog() {} // Construtor privado

    public static SistemaLog getInstance() {
        if (instancia == null) instancia = new SistemaLog();
        return instancia;
```

```
    }
    public void registrar(String acao) { ... }
}
```

4 Cenários de Uso (Exemplos de Fluxo)

4.1 Cenário 1: Triagem com Prioridade

Ação:

1. Paciente A chega às 14:00 com "Dor de cabeça"(Classificado: VERDE).
2. Paciente B chega às 14:05 com "Dor no peito"(Classificado: VERMELHO).
3. O médico solicita: `fila.proximoPaciente()`.

Resultado Esperado: O sistema retorna o **Paciente B**, mesmo ele tendo chegado depois, pois a `PriorityQueue` reordenou baseado na gravidade implementada no `compareTo`.

4.2 Cenário 2: Tentativa de Fraude no Prontuário

Ação: Um desenvolvedor júnior tenta limpar o histórico de um paciente no código da interface.

```
Prontuario p = paciente.getProntuario();
List<String> lista = p.getHistorico();
lista.clear(); // Tenta apagar tudo
```

Resultado Esperado: O Java lança uma `UnsupportedOperationException` em tempo de execução, pois o método `getHistorico` retornou uma `Collections.unmodifiableList`. O dado original permanece seguro.

4.3 Cenário 3: Faturamento Recusado

Ação: Paciente tenta realizar "Ressonância"pelo convênio "SaúdeBarata".

```
try {
    conta.adicionarProcedimento("Ressonancia");
} catch (CoberturaRecusadaException e) {
    System.out.println("Erro: " + e.getMessage());
    // Redirecionar para pagamento particular?
}
```

Resultado Esperado: O sistema captura a exceção customizada obrigatória (Checked Exception) e força o programador a tratar o erro (ex: oferecendo pagar no particular).